

CSC 648 / CSC 848

Section 03

Milestone 2 — Version 2 (M2v2)

Project: **Home4U**

AI-Assisted Interior Design Recommendation Platform

Team Number: **4**

Team Alias: **Vibecoding for Internship**

Team Members and Roles

Caleb Ponce — Team Lead, Backend/Auth, Deployment

Mason Lee — Frontend/UI

Christopher Quach — Backend/API & Database Alignment

Tyler Morris — QA / Smoke Tests / Deployment Verification

Dias Almat — Data (Styles/Tags/Search) & Recommendations

Team Lead Email

cponce8@sfsu.edu

Date

March 4, 2026

Revision History

Version	Date	Author(s)	Description
M2v2	Mar 5th, 2026	Caleb Ponce	Revised Milestone 2 submission, edited the team contributions/scores, made sure it met demands, and submitted a new one.
M2v1	Mar 4, 2026	Team	Final Milestone 2 submission
M2v0.9	Mar 3, 2026	Team	Completed architecture diagrams, APIs, and deployment documentation
M2v0.7	Mar 2, 2026	Team	Drafted data definitions and functional requirements
M2v0.5	Mar 1, 2026	Team	Created Milestone 2 document structure
M1v2	Feb 19, 2026	Team	Revised Milestone 1 based on instructor feedback
M1v1	Feb 8, 2026	Team	Initial Milestone 1 submission
M1v0.1	Feb 1, 2026	Team	Initial project draft

Table of Contents

CSC 648 / CSC 848	1
Section 03	1
Milestone 2 — Version 1 (M2v1)	1
Team Members and Roles	1
Team Lead Email	1
Date	1
Revision History	1
Table of Contents	2
1. Data Definitions	3
User	3
RoomProject	4
Style	4
Tag	4
StyleTag	5
RoomTag	5
ResemblanceScore	5
Recommendation	6
ProductItem	6
2. Prioritized Functional Requirements	6
Priority 1 — Critical	6
Priority 2 — Important	7
Priority 3 — Nice to Have	7
3. Mockups and Storyboards	8
Primary User Workflow	8
Login Page	8
Dashboard	9
Create Room Project	9
Recommendations Page	10
4. High-Level System Design	10
4.1 System Architecture Overview	10
4.2 Backend Technologies	11
4.3 Network and Deployment Design	12
4.4 High-Level APIs and Algorithms	13
4.5 Database Architecture	14
Entity Relationship Diagram	14
5. Key Project Risks	15
6. Project Management	16
7. Team Contributions	16
Caleb Ponce — Team Lead, Backend/Auth, Deployment	17

Mason Lee — Frontend/UI Development	17
Christopher Quach — Backend API and Database Integration	17
Tyler Morris — Quality Assurance and Deployment Verification	18
Dias Almat — Data Organization and Recommendation Logic	18

1. Data Definitions

The following entities represent the core database schema used by the Home4U backend.

User

Represents a registered user of the platform.

Attributes

user_id
email
password_hash
created_at

Relationship:

User → RoomProject

RoomProject

Represents a user-created room design project.

Attributes

project_id
user_id (FK → User)
room_type
budget
photo_url
created_at

Relationship:

RoomProject → RoomTag
RoomProject → Room Dimension
RoomProject → ResemblanceScore
RoomProject → Recommendation
RoomProject → RoomFurniture
RoomProject → RoomObject

Style

Represents an interior design style available for recommendation.

Attributes

style_id
name
description

Relationship:

Style → StyleTag
Style → ProductItem
Style → ResemblanceScore

Tag

Represents descriptive keywords used to categorize styles.

Attributes

tag_id
name

Relationship:

Tag → RoomTag
Tag → StyleTag

StyleTag

Associates tags with styles.

Attributes

styletag_id
style_id
tag_id
weight

RoomTag

Associates tags with a room project.

Attributes

roomtag_id
room_project_id
tag_id
is_confirmed

ResemblanceScore

Stores similarity scores between room projects and styles.

Attributes

resemblancescore_id
room_project_id
style_id
score_value
created_at

Recommendation

Stores design recommendations generated for a project.

Attributes

recommendation_id
room_project_id
description
priority_score
estimated_cost
is_completed
created_at

Relationship:

Recommendation → RoomProject

ProductItem

Represents product suggestions associated with a style.

Attributes

product_id
style_id
vendor_id
name
category
estimated_cost
url
Image_url

Relationship:

ProductItem → Style
ProductItem → Vendor
ProductItem → BudgetAllocation

BudgetPlan

Represents the budget associated with the project

Attributes

budget_id
project_id
total_budget
currency
plan_name
created_at

Relationship:

BudgetPlan → RoomProject
BudgetPlan → BudgetAllocation

BudgetAllocation

Represents how the budget is being spent

Attributes

allocation_id
budget_id
product_id
category
allocated_amount
quantity
priority_rank
created_at

Relationship:

BudgetAllocation → ProductItem

BudgetAllocation → BudgetPlan

2. Prioritized Functional Requirements

Priority 1 — Critical

FR1 — User Signup

Users shall be able to register using an email and password.

FR2 — User Login

Users shall authenticate and receive a JWT token.

FR3 — Create Room Project

Authenticated users shall create room design projects.

FR4 — List Room Projects

Users shall retrieve projects associated with their account.

FR5 — Retrieve Styles

Users shall retrieve available interior design styles.

FR6 — Retrieve Style Tags

Users shall retrieve tags associated with styles.

FR7 — Search Styles

Users shall search styles using text queries.

FR8 — Generate Recommendations

Users shall receive recommendations associated with a project.

FR9 — Enforce Project Ownership

The system shall ensure that users can access only the room projects associated with their own account.

FR10 — Validate User Credentials

The system shall validate user credentials before granting access to protected resources.

FR11 — Persist Room Project Data

The system shall store room project information in the database for later retrieval.

FR12 — Associate Recommendations with Styles

The system shall associate each generated recommendation with a style.

Priority 2 — Important

FR13 — Update Room Project

Users shall modify project information.

FR14 — Delete Room Project

Users shall delete projects.

FR15 — Upload Room Photo

Users shall upload a room image which is stored locally.

FR16 — Assign Tags to Room Project

Users shall associate tags with projects.

FR17 — Compute Resemblance Scores

The system shall calculate similarity scores between room tags and style tags.

FR18 — Mark Recommendation Complete

Users shall mark recommendations as completed.

FR19 — Update Recommendation Status

Users shall update the status of a recommendation.

FR20 — Retrieve Tags

Users shall retrieve the set of available tags.

FR21 — Search Tags

Users shall search tags using text queries.

FR22 — Associate Style Tags with Styles

The system shall associate tags with styles.

FR23 — Support Multiple Room Photos per Project

The system shall allow a room project to have multiple uploaded room photos.

Priority 3 — Nice to Have

FR24 — Product Suggestions

The system may recommend purchasable furniture products.

FR25 — AI Tag Suggestions

Future versions may include automated tag generation using computer vision.

3. Mockups and Storyboards

This section presents simplified wireframe layouts representing the major pages of the Home4U platform.

Primary User Workflow

Login



Dashboard



Create Room Project



Upload Room Photo

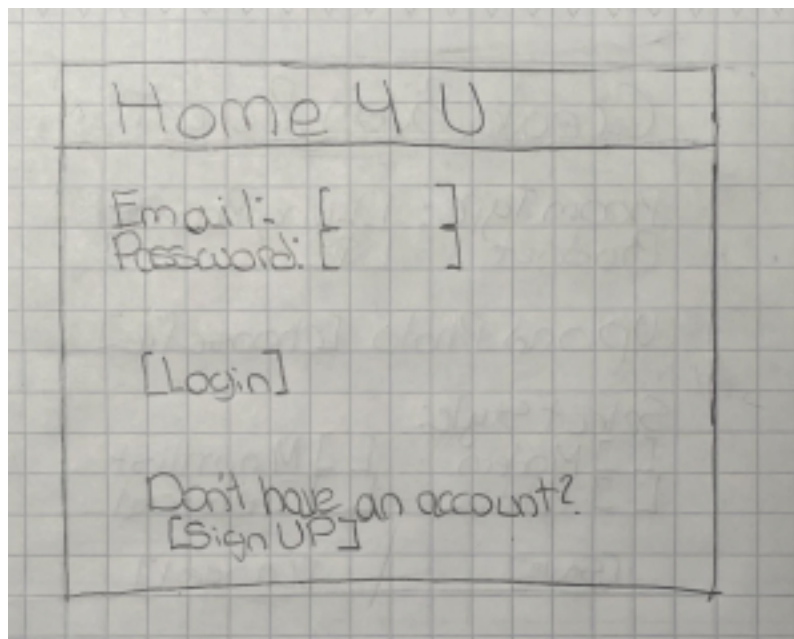


Search/Browse Styles



View Recommendations

Login Page



Dashboard

A hand-drawn sketch of a dashboard interface on a grid background. The dashboard has a title bar with 'Dashboard' on the left and '[Logout]' on the right. Below the title bar, it says 'Welcome, User'. Underneath is a button labeled '[+ Create Room Project]'. A section titled 'Projects:' contains a list of three items: '- Living Room' with a budget of '\$ 2000' and a '[View]' link, '- Bedroom' with a budget of '\$ 1500' and a '[View]' link, and '- Office' with a budget of '\$ 1200' and a '[View]' link.

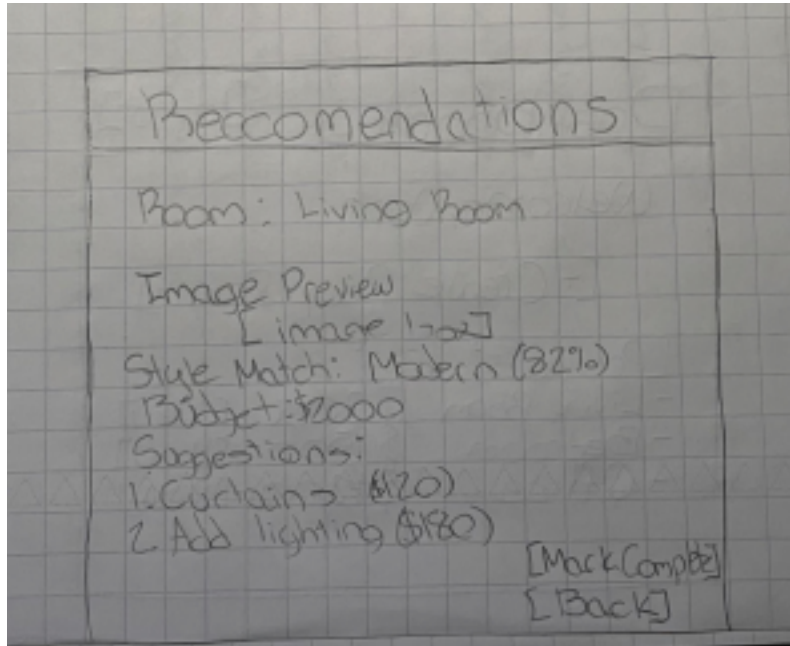
Dashboard		
Welcome, User		
[+ Create Room Project]		
Projects:		
- Living Room	\$ 2000	[View]
- Bedroom	\$ 1500	[View]
- Office	\$ 1200	[View]

Create Room Project

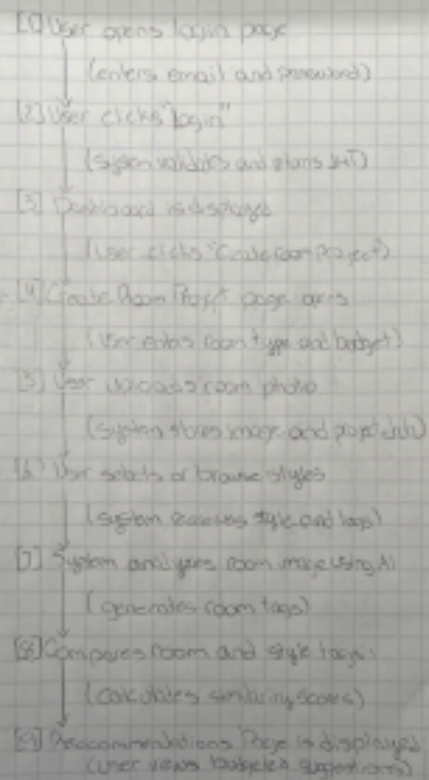
A hand-drawn sketch of a 'Create Room Project' form on a grid background. The form has a title bar with 'Create Room Project'. Below the title bar, there are three input fields: 'Room Type:' with a dropdown menu showing 'Living Room' and a downward arrow, 'Budget' with a text input field containing '\$', and 'Upload Photo' with a button labeled '[choose file]'. Below these fields is a section titled 'Select Style:' containing four checkboxes: 'Modern', 'Minimalist', 'Rustic', and 'Industrial'. At the bottom of the form are two buttons: '[Save]' and '[Cancel]'.

Create Room Project	
Room Type:	[Living Room ▼]
Budget	[\$]
Upload Photo	[choose file]
Select Style:	
☐ Modern	☐ Minimalist
☐ Rustic	☐ Industrial
[Save]	[Cancel]

Recommendations Page



Storyboard



4. High-Level System Design

4.1 System Architecture Overview

Home4U follows a **client–server architecture** in which the frontend application communicates with the backend through REST APIs. The architecture separates the user interface, application logic, and data storage layers to improve scalability and maintainability.

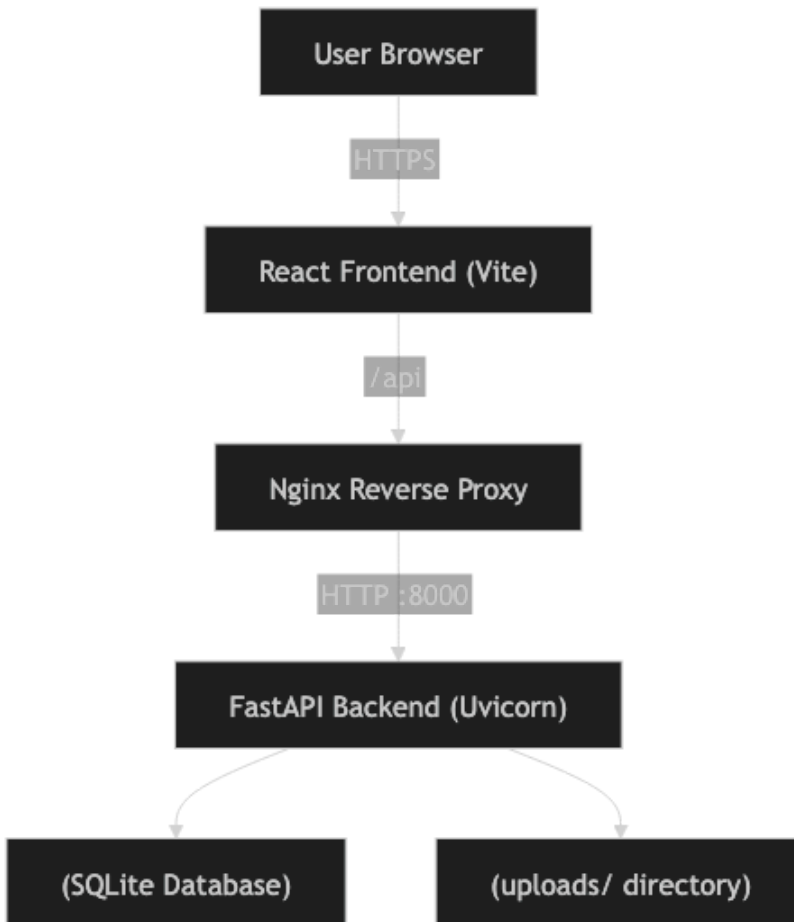
The frontend application is implemented using **React with the Vite build system**, which communicates with the backend through HTTP requests. These requests are routed through **Nginx**, which acts as a reverse proxy and handles routing and request forwarding to the FastAPI backend.

The backend service is implemented using **FastAPI**, a high-performance Python web framework. The backend processes requests, performs authentication using JWT tokens, executes search and recommendation algorithms, and communicates with the database.

Data persistence for the vertical prototype is handled using **SQLite**, which stores user data, projects, styles, tags, and recommendations. Uploaded room images are stored in a local **uploads directory** referenced by the database.

Future versions of the system will migrate the database to **PostgreSQL** and store media files in **AWS S3** to improve scalability and reliability.

Figure 1 — Home4U System Architecture



4.2 Backend Technologies

The backend of the Home4U system is implemented using modern web technologies designed for scalability and maintainability.

Backend Framework

FastAPI (Python) is used to implement the REST API services. FastAPI provides automatic request validation, asynchronous support, and high performance.

Authentication

User authentication is implemented using:

- bcrypt password hashing
- JWT authentication tokens (HS256)
- case-insensitive email normalization

These mechanisms ensure that user credentials are securely stored and verified.

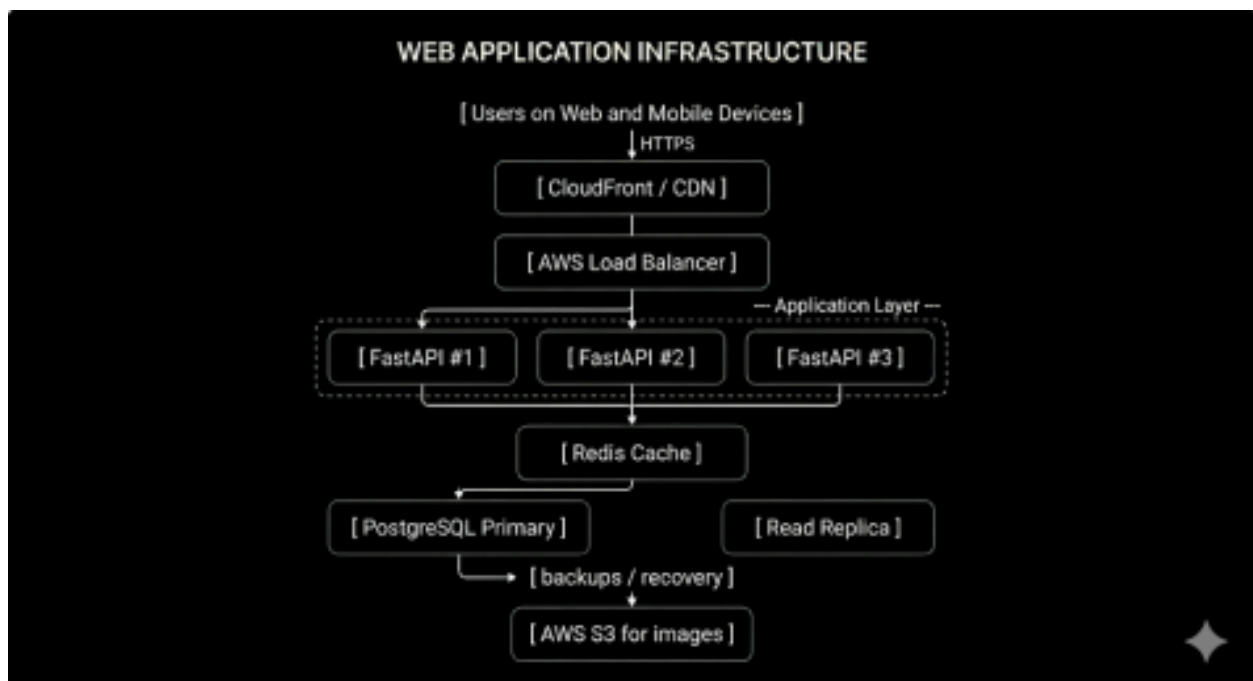
Database

For the vertical prototype, the system uses **SQLite**, which allows rapid development and simplified deployment. SQLite stores all core entities including users, projects, styles, tags, and recommendations.

Future production versions will migrate to **PostgreSQL**, which provides stronger concurrency support and improved scalability.

Media Storage

Room images uploaded by users are stored locally in an **uploads directory** on the server. The database stores a reference to the image location using the `photo_url` field.



Scalability Architecture Plan.

For the current vertical prototype, Home4U can run on a single backend instance. As traffic increases, the system can scale horizontally by placing a load balancer in front of multiple FastAPI application servers. User-uploaded room images should be moved from local server storage to AWS S3 so that media remains available across all backend instances. The database should migrate from SQLite to PostgreSQL because PostgreSQL provides stronger concurrency control, better support for multiple simultaneous users, and easier replication for reliability. A Redis cache can be added to reduce repeated database reads for frequently accessed style, tag, and recommendation data. This design allows the system to grow from a single-instance prototype to a multi-instance production deployment while improving performance, reliability, and fault tolerance.

4.3 Network and Deployment Design

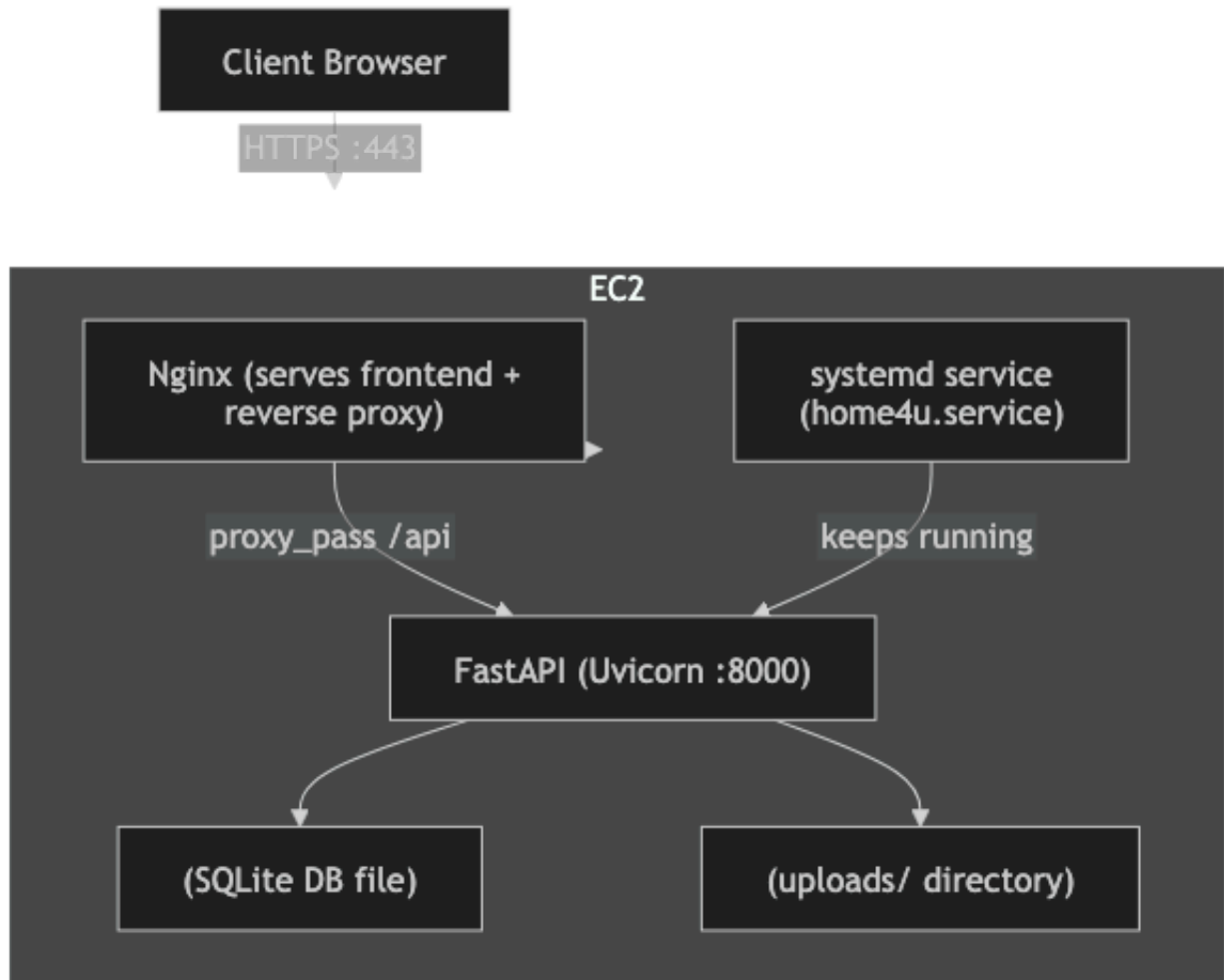
The Home4U platform is deployed on an **AWS EC2 instance running Ubuntu**. The deployment architecture uses a reverse proxy configuration to separate web traffic from application logic.

Incoming HTTP or HTTPS requests are received by **Nginx**, which serves the frontend application and forwards API requests to the FastAPI backend running on port 8000.

The backend service is managed by a **systemd service**, which ensures that the FastAPI server automatically restarts if the process fails or if the system reboots.

This architecture provides a simple but reliable infrastructure suitable for the vertical prototype while allowing future scaling.

Figure 2 — Home4U Deployment Architecture



4.4 High-Level APIs and Algorithms

The system uses the OpenAI Vision API to analyze a user's uploaded room images, using the api it then analyzes the user current room and generates descriptive tags representing the rooms visual features such as furniture types, colors, layout, and lighting. These tags are reviewed and validated before being used as structured inputs for style comparison and recommendation generation.

The weighted resemblance scoring algorithm is used to evaluate the similarity between a user's room and selected interior styles. The algorithm compares validated room tags against predefined style characteristics for each specific style such as modern, industrial, etc. Each matching feature contributes to a weighted score based on its visual importance, producing an overall similarity value that reflects how closely the room matches each selected style.

This similarity score is used to determine which styles most closely align with the current room configuration and to guide the generation of targeted improvement recommendations.

The system uses a budget-aware prioritization algorithm to generate recommendations that remain within a user-defined spending limit. Based on the calculated similarity scores, the system identifies potential room improvements that increase resemblance to the selected style. Each recommendation has an estimated cost to ensure that we do not go over the predefined budget if applicable.

4.5 Database Architecture

The Home4U database schema is designed using a normalized relational structure. The schema includes entities for users, room projects, styles, tags, recommendations, and product items.

Join tables such as **RoomTag** and **StyleTag** are used to represent many-to-many relationships between projects, styles, and tags.

Similarity between room projects and styles is represented using the **ResemblanceScore** entity, which stores computed similarity scores used by the recommendation system.

Initial Database Requirements

The database design is driven by the Priority 1 functional requirements of the system, including user authentication, room project creation, project retrieval, style browsing, and recommendation generation. The database must support storing user accounts, associating users with their room projects, and maintaining relationships between room features and design styles through tags. It must also support retrieving recommendations based on computed similarity scores while ensuring that each user can only access their own data.

DBMS Selection and Justification

For the current vertical prototype, the system uses SQLite due to its simplicity, lightweight setup, and ease of integration during development. SQLite allows rapid iteration without requiring complex configuration. However, for production deployment, the system will migrate to PostgreSQL. PostgreSQL is better suited for handling concurrent users, larger datasets, and more complex queries. It also provides stronger data integrity, indexing capabilities, and support for replication, making it more appropriate for a scalable production environment.

Media Storage Strategy

In the current implementation, uploaded room images are stored locally in an uploads directory on the server, and their file paths are stored in the database. While this approach is sufficient

for a single-instance prototype, it does not scale well in distributed environments. In future versions, media storage will be migrated to AWS S3, which provides scalable, durable, and centralized storage accessible by multiple backend instances. This change will improve system reliability and support horizontal scaling.

Data Integrity and Relationships

The database enforces relationships between entities using foreign keys to maintain data consistency. For example, each RoomProject is associated with a valid User, and recommendations are linked to specific projects. Join tables such as RoomTag and StyleTag ensure proper mapping between tags, styles, and room projects. These relationships support accurate similarity scoring and recommendation generation while maintaining referential integrity across the system.

Future Improvements

Future enhancements to the database architecture include adding indexing to frequently queried fields such as style names and tags, implementing caching mechanisms for faster retrieval of recommendation data, and enabling database replication for improved availability and fault tolerance. These improvements will allow the system to handle increased user traffic and data volume more efficiently.

Entity Relationship Diagram

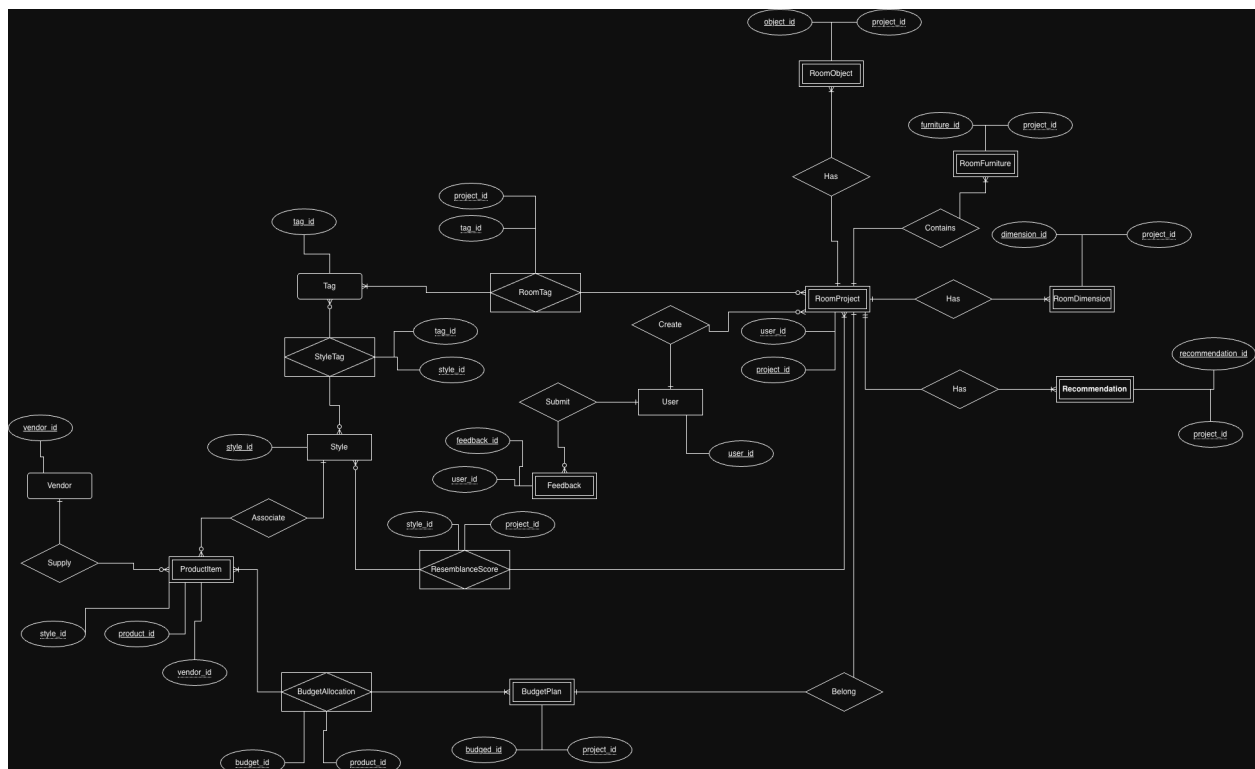


Figure 3 — Home4U Database Entity Relationship Diagram

5. Key Project Risks

Several potential risks could impact the development and deployment of the Home4U system.

Data Consistency Risks

Differences between frontend data handling and backend database schemas could result in inconsistent data states. This risk is mitigated by enforcing consistent API contracts and validating data using backend schemas.

Authentication Security Risks

Improper handling of authentication tokens could expose the system to security vulnerabilities. To mitigate this risk, the system uses bcrypt password hashing and JWT tokens with expiration times.

Single-Instance Infrastructure Risk

The vertical prototype currently runs on a single EC2 instance using SQLite. This creates a potential risk of downtime or data loss. Future versions will migrate to PostgreSQL and distributed cloud infrastructure.

Media Storage Growth

As more users upload room images, storage requirements will increase. Future versions of the system will migrate media storage to scalable cloud storage services such as AWS S3.

Team Coordination Risk

As multiple team members contribute to the same codebase, integration issues may arise. This risk is mitigated by using GitHub version control, pull requests, and regular team meetings.

6. Project Management

Development Tools

GitHub — version control

Notion — project tracking

Discord — team communication

Development Process

Trunk-based development with small pull requests and code reviews.

Definition of Done

Feature merged to repository

Deployment successful

Smoke tests pass

Core workflow operational

7. Team Contributions

Caleb Ponce — Team Lead, Backend/Auth, Deployment (Score: 10/10)

Caleb Ponce served as the Team Lead and ensured alignment across all milestone requirements, coordinating both technical execution and documentation quality.

On the technical side, Caleb implemented and maintained the connectivity between the frontend (React/Vite) and backend (FastAPI), ensuring reliable API communication across authentication and project workflows. He also refined the UI/UX of the platform to improve usability, navigation flow, and overall user experience, making the interface more intuitive and production-ready. Additionally, Caleb debugged and resolved frontend issues that impacted functionality and user interaction.

Beyond implementation, Caleb oversaw system integration, ensured consistency across diagrams and architecture, and supported team members to maintain a cohesive and polished final submission.

Mason Lee — Frontend/UI Development (Score: 10/10)

Mason Lee led the development of the frontend interface and collaborated closely on design architecture.

He worked with Christopher to design and refine frontend mockups and storyboards, ensuring consistency between UI components and system functionality. Mason contributed to structuring the layout of key pages such as authentication views, dashboard, and project interfaces, while aligning frontend behavior with backend expectations.

His work ensured that the frontend design was not only visually structured but also aligned with the overall system architecture and user flow.

Christopher Quach — Frontend Architecture & Backend Alignment (Score: 10/10)

Christopher Quach collaborated with Mason on frontend architecture, focusing on aligning system design with implementation.

He contributed to developing mockups and storyboards while ensuring that frontend structures were consistent with backend API design and database relationships. Christopher played a key

role in bridging frontend and backend logic, ensuring that components, data flow, and system interactions were coherent.

His contributions strengthened the architectural consistency between UI design and backend systems.

Tyler Morris — QA, Feedback Implementation, and Diagram Refinement (Score: 10/10)

Tyler Morris focused on quality assurance and directly implemented professor feedback to improve the project.

He revised and corrected system diagrams, specifically refining the Nginx reverse proxy and backend architecture diagrams to better reflect actual system behavior and deployment flow. Tyler ensured that diagrams were accurate, consistent, and aligned with the implemented architecture.

Additionally, he validated system functionality and ensured that revisions addressed prior feedback, improving the overall quality and correctness of the milestone.

Dias Almat — Data Design, ERD Refinement, and Diagram Quality Control (Score: 10/10)

Dias Almat played a critical role in refining the data architecture and improving documentation quality.

He revised and corrected the ERD diagram, ensuring accurate representation of entity relationships such as Style, Tag, RoomProject, and their associative tables. Dias also helped restructure architectural components where necessary to better reflect system design.

Beyond his individual contributions, he supported other team members by reviewing and improving diagrams across sections, ensuring visual consistency, clarity, and correctness throughout the documentation.